Day 8 Task Workflow — Tuesday, Mar 4, 2026

> **Sprint:** 2 (Capture Engine + Core UI) | **Phase:** Capture Refinement + ML Prep
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Capture engine hardening, NSL-KDD feature validation, ML pipeline scaffolding, Docker cleanup
> **Grade:** Week 2 Day 1 A COMPLETE ☑ | Week 2 Day 2 STARTING ⬤

Day 8 Objective

Continue Week 2 by hardening the capture engine and preparing the ML pipeline foundation so that by end of day:

- Capture engine handles all edge cases gracefully (malformed packets, memory pressure, reconnection)
- Feature extraction validated against NSL-KDD 41-feature spec with mapping documented
- Missing NSL-KDD features (land, wrong_fragment, urgent, content/time/host-based) added to extractor
- `backend/ml/` directory structure scaffolded per MASTER_DOC_PART5 §2.1
- NSL-KDD dataset downloaded and initial loader implemented
- Docker Compose cleaned up (remove deprecated `version` key)
- Data pipeline verified end-to-end: Scapy → FlowAggregator → FeatureExtractor → Redis → PostgreSQL

> **NOTE:** Frontend tasks (War Room data connection, WebSocket wiring, component integration) have been **deferred to Full-Stack Dev** per `docs/FRONTEND_TASKS_DAY8.md`. This document covers **Lead Architect tasks only.**

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope. No Kafka, Kubernetes, Elasticsearch, or overengineered infrastructure.

Requirement	Source Document	Section
Feature vector compatibility with NSL-KDD	MASTER_DOC_PART4	§3.1 (Feature Categories)
NSL-KDD dataset preparation pipeline	MASTER_DOC_PART4	§2.1 (Primary Dataset)
ML directory structure	MASTER_DOC_PART5	§2.1 (Repository Structure)
Capture engine robustness	MASTER_DOC_PART2	§2.1 (Tier 1: Capture Engine)
Docker Compose service config	MASTER_DOC_PART2	§3.2 (Docker Compose)
Feature extraction from Scapy	MASTER_DOC_PART4	§3.2 (Feature Extraction)
Dataset preparation pipeline	MASTER_DOC_PART4	§2.4 (Dataset Preparation)

Architectural Constraints

> **ZERO TOLERANCE for deviation.** These are locked decisions from the master documentation.

Constraint	Rationale	Enforcement
Python 3.11+ only	Stack locked	requirements.txt
scikit-learn for IF + RF	Stack locked	MASTER_DOC_PART4 §4-5
TensorFlow/Keras for AE	Stack locked	MASTER_DOC_PART4 §6
NSL-KDD primary dataset	Academic requirement	MASTER_DOC_PART4 §2.1
41 features minimum	NSL-KDD compatibility	Feature extractor
pandas for data prep	Stack locked	MASTER_DOC_PART4 §2.4
Scapy only for capture	No tcpdump/libpcap wrappers	Code review
PostgreSQL JSONB for features	Schema locked	MASTER_DOC_PART2 §4.2

Task Breakdown

TASK 1 — Capture Engine Hardening 🟤

Time Est: 90 min | **Priority:** 🟤 Critical

Harden the capture engine to handle edge cases, improve resilience, and add production-grade error handling.

1.1 Malformed Packet Handling (`backend/capture/engine.py`)

The current `_extract_packet` method handles IP/TCP/UDP/ICMP but needs hardening for:

Edge cases to handle:

```
# In CaptureEngine._extract_packet():

def _extract_packet(self, packet: Packet) -> Optional[PacketRecord]:
    """Extract relevant data from Scapy packet into PacketRecord."""
    try:
        if not packet.haslayer(IP):
            return None

        ip_layer = packet[IP]

        # Guard: skip malformed IP
        if not hasattr(ip_layer, 'src') or not hasattr(ip_layer, 'dst'):
            return None

        # Guard: skip multicast/broadcast
        if ip_layer.dst.startswith('224.') or ip_layer.dst == '255.255.255.255':
            return None

        timestamp = time.time()
        length = len(packet)
```

```
        # Existing TCP/UDP/ICMP extraction...
        # (keep current implementation)

    except Exception as exc:
        logger.debug("[Engine] Packet parse error: %s", exc)
        return None
```

1.2 Redis Reconnection Logic (`backend/capture/publisher.py`)

Add automatic reconnection if Redis connection drops:

```
async def publish_flow(self, flow_record: dict) -> None:
    """Publish flow with automatic reconnection."""
    for attempt in range(3):
        try:
            if self._redis is None or self._redis.connection is None:
                await self.connect()
            await self._redis.publish(
                self.flow_channel,
                json.dumps(flow_record, default=str)
            )
            return
        except (ConnectionError, redis.ConnectionError) as exc:
            logger.warning("[Publisher] Redis connection lost (attempt %d/3): %s",
attempt + 1, exc)
            self._redis = None
            if attempt < 2:
                await asyncio.sleep(1 * (attempt + 1))  # Backoff: 1s, 2s
    logger.error("[Publisher] Failed to publish after 3 attempts")
```

1.3 Memory Pressure Monitoring (`backend/capture/engine.py`)

Add flow buffer memory monitoring to the stats loop:

```
async def _stats_loop(self) -> None:
    """Periodically report capture statistics."""
    while self.running:
        try:
            await asyncio.sleep(self.config.stats_interval)
            status = self.get_status()
            pps = status["packets_captured"] / max(status["uptime_seconds"], 1)

            # Memory check: warn if flow buffer is > 80% full
            buffer_usage = status["active_flows"] / self.config.max_flows_buffer
            if buffer_usage > 0.8:
                logger.warning(
                    "[Engine] Flow buffer at %.0f%% capacity (%d/%d)",
                    buffer_usage * 100,
```

```
                    status["active_flows"],
                    self.config.max_flows_buffer,
                )

            logger.info(
                "[Engine] Stats: %d pkts | %d flows | %d published | %.1f pps | %d
active (%.0f%%)",
                status["packets_captured"],
                status["flows_completed"],
                status["flows_published"],
                pps,
                status["active_flows"],
                buffer_usage * 100,
            )

            await self.publisher.publish_system_status(status)
        except asyncio.CancelledError:
            break
        except Exception as exc:
            logger.error("[Engine] Stats error: %s", exc)
```

Verification:

Check	Command	Expected
Malformed packet skipped	Send malformed packet	No crash, debug log
Multicast filtered	Send to 224.x.x.x	Packet ignored
Redis reconnect	Restart Redis, wait	Auto-reconnects within 3s
Buffer warning	Fill 8000+ flows	Warning log at 80%
Graceful shutdown	SIGTERM to capture	Clean shutdown message

TASK 2 — NSL-KDD Feature Mapping & Validation 🟤

Time Est: 120 min | **Priority:** 🟤 Critical

The current `FeatureExtractor` produces 30 features. NSL-KDD requires 41 features. We need to add the missing features and document the mapping.

2.1 NSL-KDD Feature Gap Analysis

Current features (30): duration, protocol_type, service, flag, src_bytes, dst_bytes, total_bytes, byte_ratio, src_packets, dst_packets, total_packets, packet_ratio, mean_iat, std_iat, min_iat, max_iat, syn_count, ack_count, fin_count, rst_count, psh_count, urg_count, payload_entropy, mean_payload_size, has_payload, packets_per_second, bytes_per_packet, bytes_per_second, is_internal, port_class

Missing NSL-KDD features (11):

#	Feature	Type	NSL-KDD Column	How to Compute from Live Traffic

#	Feature	Type	NSL-KDD Column	How to Compute from Live Traffic
1	`land`	bool	land	`src_ip == dst_ip and src_port == dst_port`
2	`wrong_fragment`	int	wrong_fragment	Count IP fragments with errors (MF flag + offset check)
3	`urgent`	int	urgent	Count of URG-flagged packets (already have `urg_count`)
4	`hot`	int	hot	Indicators of "hot" access (approximate: non-standard port access count)
5	`num_failed_logins`	int	num_failed_logins	Set to 0 (requires DPI — approximate only)
6	`logged_in`	bool	logged_in	Approximate: connection established (SYN+ACK seen)
7	`num_compromised`	int	num_compromised	Set to 0 (requires DPI)
8	`root_shell`	bool	root_shell	Set to 0 (requires DPI)
9	`su_attempted`	bool	su_attempted	Set to 0 (requires DPI)
10	`num_root`	int	num_root	Set to 0 (requires DPI)
11	`num_file_creations`	int	num_file_creations	Set to 0 (requires DPI)

> **NOTE:** Features 5-11 are "content features" that require deep packet inspection of application-layer payloads. NSL-KDD extracted these from tcpdump connection records. For live traffic, we approximate with 0 and rely on network-level features. This is standard practice in modern IDS research — see CICIDS2017 which dropped content features entirely.

2.2 Add Missing Features to Extractor (`backend/capture/feature_extractor.py`)

Add the following to the `extract()` method after the existing behavioral features:

```
# ── NSL-KDD Compatibility Features ──────────────────────
# Land attack detection
features["land"] = (
    flow.key.src_ip == flow.key.dst_ip and
    flow.key.src_port == flow.key.dst_port and
    flow.key.src_port != 0
)

# Wrong fragments (approximated from IP layer — requires packet-level data)
features["wrong_fragment"] = 0  # Would need IP fragment tracking

# Urgent packets (mapped from urg_count)
features["urgent"] = flow.urg_count

# ── Content Features (approximated — no DPI) ────────────
```

```
# These require application-layer inspection not available from
# Scapy flow-level aggregation. Set to 0 per standard practice.
# NSL-KDD extracted these from BSM audit logs, not raw packets.
features["hot"] = 0
features["num_failed_logins"] = 0
features["logged_in"] = 1 if (flow.syn_count > 0 and flow.ack_count > 0) else 0
features["num_compromised"] = 0
features["root_shell"] = 0
features["su_attempted"] = 0
features["num_root"] = 0
features["num_file_creations"] = 0
features["num_shells"] = 0
features["num_access_files"] = 0
features["is_host_login"] = 0
features["is_guest_login"] = 0
```

2.3 Add Time-Based Traffic Features (2-Second Window)

Per MASTER_DOC_PART4 §3.1 features 16-22, these require a **connection history window**. Add a `ConnectionTracker` class:

```
# New file or addition to feature_extractor.py

from collections import deque
from dataclasses import dataclass
import time

@dataclass
class ConnectionRecord:
    """Minimal record for time-window features."""
    timestamp: float
    dst_ip: str
    dst_port: int
    service: str
    flag: str  # SF, S0, REJ, etc.

class ConnectionTracker:
    """
    Tracks recent connections for NSL-KDD time-based and host-based features.

    Per MASTER_DOC_PART4 §3.1:
    - Time features: 2-second window of connections to same host
    - Host features: 100-connection window to same destination
    """

    def __init__(self, time_window: float = 2.0, host_window: int = 100):
        self.time_window = time_window
        self.host_window = host_window
        self._history: deque = deque(maxlen=10000)

    def add_connection(self, record: ConnectionRecord) -> None:
```

```
        """Add a completed connection to history."""
        self._history.append(record)

    def get_time_features(self, current: ConnectionRecord) -> dict:
        """Compute NSL-KDD time-based features (2-second window)."""
        cutoff = current.timestamp - self.time_window
        window = [r for r in self._history if r.timestamp >= cutoff]

        same_host = [r for r in window if r.dst_ip == current.dst_ip]
        same_srv = [r for r in window if r.service == current.service]

        count = len(same_host)
        srv_count = len(same_srv)

        # Error rates
        serror = sum(1 for r in same_host if r.flag in ("S0", "S1", "S2", "S3"))
        rerror = sum(1 for r in same_host if r.flag == "REJ")
        srv_serror = sum(1 for r in same_srv if r.flag in ("S0", "S1", "S2",
"S3"))
        srv_rerror = sum(1 for r in same_srv if r.flag == "REJ")

        return {
            "count": count,
            "srv_count": srv_count,
            "serror_rate": serror / max(count, 1),
            "srv_serror_rate": srv_serror / max(srv_count, 1),
            "rerror_rate": rerror / max(count, 1),
            "srv_rerror_rate": srv_rerror / max(srv_count, 1),
            "same_srv_rate": sum(1 for r in same_host if r.service ==
current.service) / max(count, 1),
            "diff_srv_rate": 1.0 - (sum(1 for r in same_host if r.service ==
current.service) / max(count, 1)),
        }

    def get_host_features(self, current: ConnectionRecord) -> dict:
        """Compute NSL-KDD host-based features (100-connection window)."""
        recent = list(self._history)[-self.host_window:]

        dst_host = [r for r in recent if r.dst_ip == current.dst_ip]
        dst_host_srv = [r for r in dst_host if r.service == current.service]

        dst_host_count = len(dst_host)
        dst_host_srv_count = len(dst_host_srv)

        serror = sum(1 for r in dst_host if r.flag in ("S0", "S1", "S2", "S3"))
        rerror = sum(1 for r in dst_host if r.flag == "REJ")
        srv_serror = sum(1 for r in dst_host_srv if r.flag in ("S0", "S1", "S2",
"S3"))
        srv_rerror = sum(1 for r in dst_host_srv if r.flag == "REJ")

        return {
            "dst_host_count": dst_host_count,
            "dst_host_srv_count": dst_host_srv_count,
            "dst_host_same_srv_rate": dst_host_srv_count / max(dst_host_count, 1),
```

```
            "dst_host_diff_srv_rate": 1.0 - (dst_host_srv_count /
max(dst_host_count, 1)),
            "dst_host_same_src_port_rate": sum(1 for r in dst_host if r.dst_port
== current.dst_port) / max(dst_host_count, 1),
            "dst_host_serror_rate": serror / max(dst_host_count, 1),
            "dst_host_srv_serror_rate": srv_serror / max(dst_host_srv_count, 1),
            "dst_host_rerror_rate": rerror / max(dst_host_count, 1),
            "dst_host_srv_rerror_rate": srv_rerror / max(dst_host_srv_count, 1),
            "dst_host_srv_diff_host_rate": 0.0,  # Requires cross-host tracking
        }
```

2.4 Update Feature Names List

Update `feature_names()` in `FeatureExtractor` to include all 41+ features:

```
def feature_names(self) -> List[str]:
    """Return ordered list of all feature names (NSL-KDD compatible +
extended)."""
    return [
        # NSL-KDD Basic (9)
        "duration", "protocol_type", "service", "flag",
        "src_bytes", "dst_bytes", "land", "wrong_fragment", "urgent",
        # NSL-KDD Content (13)
        "hot", "num_failed_logins", "logged_in", "num_compromised",
        "root_shell", "su_attempted", "num_root", "num_file_creations",
        "num_shells", "num_access_files", "is_host_login", "is_guest_login",
        # NSL-KDD Time-based (9) — requires ConnectionTracker
        "count", "srv_count", "serror_rate", "srv_serror_rate",
        "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
        # NSL-KDD Host-based (10) — requires ConnectionTracker
        "dst_host_count", "dst_host_srv_count", "dst_host_same_srv_rate",
        "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
        "dst_host_serror_rate", "dst_host_srv_serror_rate",
        "dst_host_rerror_rate", "dst_host_srv_rerror_rate",
        "dst_host_srv_diff_host_rate",
        # Extended features (CICIDS2017 compatible)
        "total_bytes", "byte_ratio",
        "src_packets", "dst_packets", "total_packets", "packet_ratio",
        "mean_iat", "std_iat", "min_iat", "max_iat",
        "syn_count", "ack_count", "fin_count", "rst_count", "psh_count",
"urg_count",
        "payload_entropy", "mean_payload_size", "has_payload",
        "packets_per_second", "bytes_per_packet", "bytes_per_second",
        "is_internal", "port_class",
    ]
```

Verification:

Check	Command	Expected

Check	Command	Expected
Feature count	`len(extractor.feature_names())`	≥ 41 features
NSL-KDD basic features present	Check feature dict	duration, protocol_type, service, flag, src_bytes, dst_bytes, land, wrong_fragment, urgent
Content features present	Check feature dict	hot, logged_in, num_failed_logins (all 0 except logged_in)
Time features computed	Extract with ConnectionTracker	count, srv_count, serror_rate, etc.
Host features computed	Extract with ConnectionTracker	dst_host_count, dst_host_srv_count, etc.
Feature names ordered	`extractor.feature_names()`	NSL-KDD order first, then extended
Land detection works	Flow with same src/dst IP+port	`land = True`

TASK 3 — ML Pipeline Directory Scaffolding 🔴

Time Est: 60 min | **Priority:** 🔴 Critical

Create the `backend/ml/` directory structure per MASTER_DOC_PART5 §2.1. This sets up the skeleton for Week 3's ML model training.

3.1 Create ML Module Structure

Files to create:

```
backend/ml/
├── __init__.py                    # ML module init
├── datasets/
│   ├── __init__.py
│   ├── nsl_kdd.py                 # NSL-KDD loader + preprocessor
│   └── cicids2017.py              # CICIDS2017 loader (Week 5)
├── models/
│   ├── __init__.py
│   ├── isolation_forest.py        # IF wrapper class
│   ├── random_forest.py           # RF wrapper class
│   └── autoencoder.py             # AE wrapper class
├── training/
│   ├── __init__.py
│   ├── train_all.py               # Orchestrate training of all models
│   ├── evaluate.py                # Evaluation metrics + reports
│   └── hyperparams.py             # Hyperparameter configurations
├── inference/
│   ├── __init__.py
```

```
│   ├── model_manager.py          # Load and manage trained models
│   ├── ensemble_scorer.py        # Composite scoring logic
│   └── worker.py                 # Redis subscriber + inference loop
└── saved_models/                 # Trained model artifacts
    └── .gitkeep
```

3.2 Create `ml/__init__.py`

```
"""
ThreatMatrix AI — Machine Learning Pipeline

Per MASTER_DOC_PART4: Three-model ensemble for anomaly detection.
- Model 1: Isolation Forest (unsupervised anomaly detection)
- Model 2: Random Forest (supervised multi-class classification)
- Model 3: Autoencoder (deep learning reconstruction-based detection)
"""
```

3.3 Create `ml/datasets/nsl_kdd.py`

```
"""
ThreatMatrix AI — NSL-KDD Dataset Loader

Per MASTER_DOC_PART4 §2.1: Primary benchmark dataset for IDS evaluation.
- 125,973 training records + 22,544 test records
- 41 features + 1 label
- 4 attack categories: DoS, Probe, R2L, U2R
"""

from __future__ import annotations

import logging
from pathlib import Path
from typing import Tuple

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

logger = logging.getLogger(__name__)

# NSL-KDD column names (41 features + 2 labels)
NSL_KDD_COLUMNS = [
    "duration", "protocol_type", "service", "flag",
    "src_bytes", "dst_bytes", "land", "wrong_fragment", "urgent",
    "hot", "num_failed_logins", "logged_in", "num_compromised",
    "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "is_host_login", "is_guest_login",
```

```
    "count", "srv_count", "serror_rate", "srv_serror_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate",
    "dst_host_count", "dst_host_srv_count", "dst_host_same_srv_rate",
    "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate", "dst_host_serror_rate",
    "dst_host_srv_serror_rate", "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate",
    "label", "difficulty_level"
]

# Attack type → category mapping
ATTACK_CATEGORIES = {
    "normal": "normal",
    # DoS attacks
    "back": "dos", "land": "dos", "neptune": "dos", "pod": "dos",
    "smurf": "dos", "teardrop": "dos", "mailbomb": "dos",
    "apache2": "dos", "processtable": "dos", "udpstorm": "dos",
    # Probe attacks
    "ipsweep": "probe", "nmap": "probe", "portsweep": "probe",
    "satan": "probe", "mscan": "probe", "saint": "probe",
    # R2L attacks
    "ftp_write": "r2l", "guess_passwd": "r2l", "imap": "r2l",
    "multihop": "r2l", "phf": "r2l", "spy": "r2l",
    "warezclient": "r2l", "warezmaster": "r2l", "snmpgetattack": "r2l",
    "named": "r2l", "xlock": "r2l", "xsnoop": "r2l",
    "sendmail": "r2l", "httptunnel": "r2l", "worm": "r2l",
    "snmpguess": "r2l",
    # U2R attacks
    "buffer_overflow": "u2r", "loadmodule": "u2r", "perl": "u2r",
    "rootkit": "u2r", "xterm": "u2r", "ps": "u2r",
    "sqlattack": "u2r",
}

DATASET_DIR = Path(__file__).parent.parent / "saved_models" / "datasets"


class NSLKDDLoader:
    """
    Load and preprocess the NSL-KDD dataset.

    Per MASTER_DOC_PART4 §2.4, the preparation pipeline:
    1. Load raw data (CSV with column headers)
    2. Handle missing values
    3. Encode categorical features (Label Encoding)
    4. Normalize numerical features (StandardScaler)
    5. Map attack labels → 5-class categories
    6. Stratified train/test split
    """

    def __init__(self, data_dir: Path | None = None):
        self.data_dir = data_dir or DATASET_DIR
        self.label_encoders: dict[str, LabelEncoder] = {}
        self.scaler: StandardScaler | None = None
```

```
        self.feature_names: list[str] = []

    def load_train(self) -> pd.DataFrame:
        """Load NSL-KDD training set (KDDTrain+.txt)."""
        train_path = self.data_dir / "KDDTrain+.txt"
        if not train_path.exists():
            raise FileNotFoundError(
                f"NSL-KDD training data not found at {train_path}. "
                f"Download from: https://www.unb.ca/cic/datasets/nsl.html"
            )
        df = pd.read_csv(train_path, header=None, names=NSL_KDD_COLUMNS)
        logger.info("Loaded NSL-KDD train set: %d records, %d columns", len(df),
len(df.columns))
        return df

    def load_test(self) -> pd.DataFrame:
        """Load NSL-KDD test set (KDDTest+.txt)."""
        test_path = self.data_dir / "KDDTest+.txt"
        if not test_path.exists():
            raise FileNotFoundError(
                f"NSL-KDD test data not found at {test_path}. "
                f"Download from: https://www.unb.ca/cic/datasets/nsl.html"
            )
        df = pd.read_csv(test_path, header=None, names=NSL_KDD_COLUMNS)
        logger.info("Loaded NSL-KDD test set: %d records, %d columns", len(df),
len(df.columns))
        return df

    def preprocess(
        self, df: pd.DataFrame, fit: bool = True
    ) -> Tuple[np.ndarray, np.ndarray, list[str]]:
        """
        Full preprocessing pipeline.

        Args:
            df: Raw NSL-KDD DataFrame
            fit: If True, fit encoders/scaler (training). If False, transform only
(test).

        Returns:
            (X, y, feature_names) — NumPy arrays ready for model training.
        """
        df = df.copy()

        # 1. Map labels to 5-class categories
        df["label"] = df["label"].str.strip().str.lower()
        df["attack_category"] =
df["label"].map(ATTACK_CATEGORIES).fillna("unknown")
        df = df[df["attack_category"] != "unknown"]  # Drop unmapped (rare)

        # 2. Drop difficulty_level (not a feature)
        df = df.drop(columns=["difficulty_level", "label"], errors="ignore")

        # 3. Separate features and target
```

```
        y_raw = df["attack_category"]
        X_df = df.drop(columns=["attack_category"])

        # 4. Encode categorical features
        categorical_cols = ["protocol_type", "service", "flag"]
        for col in categorical_cols:
            if fit:
                le = LabelEncoder()
                X_df[col] = le.fit_transform(X_df[col].astype(str))
                self.label_encoders[col] = le
            else:
                le = self.label_encoders[col]
                # Handle unseen labels in test set
                X_df[col] = X_df[col].astype(str).map(
                    lambda x, _le=le: _le.transform([x])[0]
                    if x in _le.classes_ else -1
                )

        # 5. Encode target
        if fit:
            self.label_encoders["target"] = LabelEncoder()
            y = self.label_encoders["target"].fit_transform(y_raw)
        else:
            y = self.label_encoders["target"].transform(y_raw)

        # 6. Store feature names
        self.feature_names = list(X_df.columns)

        # 7. Convert to numpy
        X = X_df.values.astype(np.float32)

        # 8. Scale numerical features
        if fit:
            self.scaler = StandardScaler()
            X = self.scaler.fit_transform(X)
        else:
            X = self.scaler.transform(X)

        logger.info(
            "Preprocessed: X=%s, y=%s, classes=%s",
            X.shape, y.shape,
            list(self.label_encoders["target"].classes_)
        )

        return X, y, self.feature_names

    def get_normal_mask(self, y: np.ndarray) -> np.ndarray:
        """Get boolean mask for normal traffic (for unsupervised training)."""
        normal_idx = list(self.label_encoders["target"].classes_).index("normal")
        return y == normal_idx

    def get_class_names(self) -> list[str]:
        """Get ordered list of class names."""
        return list(self.label_encoders["target"].classes_)
```

3.4 Create Stub Files for Remaining ML Modules

Create minimal stubs for files that will be fully implemented in Week 3:

`ml/models/isolation_forest.py`:

```
"""
ThreatMatrix AI — Isolation Forest Model Wrapper

Per MASTER_DOC_PART4 §4: Unsupervised anomaly detection.
Implementation: Week 3

Hyperparameters (per §4.3):
- n_estimators: 200
- contamination: 0.05
- max_samples: 'auto'
- random_state: 42
"""

# TODO: Implement in Week 3
```

`ml/models/random_forest.py`:

```
"""
ThreatMatrix AI — Random Forest Classifier Wrapper

Per MASTER_DOC_PART4 §5: Supervised multi-class classification.
Implementation: Week 3

Hyperparameters (per §5.2):
- n_estimators: 300
- max_depth: 30
- class_weight: 'balanced'
- random_state: 42
"""

# TODO: Implement in Week 3
```

`ml/models/autoencoder.py`:

```
"""
ThreatMatrix AI — Autoencoder Model Wrapper

Per MASTER_DOC_PART4 §6: Deep learning reconstruction-based anomaly detection.
Implementation: Week 3
```

```
    Architecture (per §6.2):
    - Encoder: 64 → 32 → 16 (bottleneck)
    - Decoder: 32 → 64 → n_features
    - Activation: relu (hidden), sigmoid (output)
    - Loss: MSE
    """

    # TODO: Implement in Week 3
```

ml/training/hyperparams.py:

```
"""
ThreatMatrix AI — Hyperparameter Configurations

Per MASTER_DOC_PART4 §4.4, §5.2, §6.3

Centralized hyperparameter configs for all three models.
"""

ISOLATION_FOREST_PARAMS = {
    "n_estimators": 200,
    "contamination": 0.05,
    "max_samples": "auto",
    "max_features": 1.0,
    "bootstrap": False,
    "random_state": 42,
    "n_jobs": -1,
}

RANDOM_FOREST_PARAMS = {
    "n_estimators": 300,
    "max_depth": 30,
    "min_samples_split": 5,
    "min_samples_leaf": 2,
    "max_features": "sqrt",
    "class_weight": "balanced",
    "criterion": "gini",
    "random_state": 42,
    "n_jobs": -1,
}

AUTOENCODER_PARAMS = {
    "encoder_layers": [64, 32, 16],
    "decoder_layers": [32, 64],
    "activation": "relu",
    "output_activation": "sigmoid",
    "dropout_rate": 0.2,
    "learning_rate": 0.001,
    "batch_size": 256,
    "epochs": 100,
    "patience": 10,
    "validation_split": 0.15,
```

```
}

ENSEMBLE_WEIGHTS = {
    "isolation_forest": 0.30,     # W_IF per PART4 §1.2
    "random_forest": 0.45,        # W_RF per PART4 §1.2
    "autoencoder": 0.25,          # W_AE per PART4 §1.2
}

ALERT_THRESHOLDS = {
    "critical": 0.90,
    "high": 0.75,
    "medium": 0.50,
    "low": 0.30,
}
```

`ml/inference/ensemble_scorer.py`:

```
"""
ThreatMatrix AI — Ensemble Scorer

Per MASTER_DOC_PART4 §1.2: Composite anomaly scoring.

Scoring Formula:
    composite = W_IF × IF_score + W_RF × RF_confidence + W_AE × AE_recon_error

Implementation: Week 3-4
"""

# TODO: Implement in Week 3
```

`ml/inference/worker.py`:

```
"""
ThreatMatrix AI — ML Inference Worker

Per MASTER_DOC_PART4 §8: Real-time inference pipeline.

Redis subscriber: listens on 'flows:live' channel,
scores each flow with the ensemble, publishes results
to 'ml:scored' channel.

Implementation: Week 4
"""

# TODO: Implement in Week 4
```

Create all `__init__.py` files as empty with module docstrings.

Verification:

Check	Command	Expected
ML directory exists	`ls backend/ml/`	5 subdirectories
All **init**.py present	`find backend/ml -name __init__.py`	5 files
NSL-KDD loader imports	`python -c "from ml.datasets.nsl_kdd import NSLKDDLoader"`	No errors
Hyperparams imports	`python -c "from ml.training.hyperparams import ENSEMBLE_WEIGHTS"`	No errors
saved_models dir exists	`ls backend/ml/saved_models/`	.gitkeep present
Structure matches PART5 §2.1	Compare to master doc	Exact match

---

TASK 4 — NSL-KDD Dataset Download 🟢

Time Est: 30 min | **Priority:** 🟢 Medium

Download the NSL-KDD dataset to the VPS for Week 3 training.

4.1 Download on VPS

```
# SSH into VPS
ssh root@187.124.45.161

# Create dataset directory
mkdir -p /root/threatmatrix-ai/backend/ml/saved_models/datasets

# Download NSL-KDD from UNB mirror
cd /root/threatmatrix-ai/backend/ml/saved_models/datasets

# Option 1: Direct download from UNB
wget https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+.txt
wget https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTest+.txt
wget
https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+_20Percent.txt
wget https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTest-21.txt

# Verify file sizes
ls -la *.txt
# Expected:
# KDDTrain+.txt          ~18MB (125,973 records)
# KDDTest+.txt           ~3.2MB (22,544 records)
# KDDTrain+_20Percent.txt ~3.6MB (25,192 records)
# KDDTest-21.txt         ~1.5MB (11,850 records)
```

```
# Quick data check
head -1 KDDTrain+.txt
# Expected: 0,tcp,http,SF,215,45076,0,0,0,0,0,1,0,0,...
wc -l KDDTrain+.txt
# Expected: 125973
```

4.2 Add to .gitignore

```
# Add dataset files to .gitignore (don't commit 18MB+ files)
echo "backend/ml/saved_models/datasets/*.txt" >> .gitignore
echo "backend/ml/saved_models/datasets/*.csv" >> .gitignore
echo "backend/ml/saved_models/*.pkl" >> .gitignore
echo "backend/ml/saved_models/*.h5" >> .gitignore
echo "backend/ml/saved_models/*.keras" >> .gitignore
```

Verification:

Check	Command	Expected
Train file exists	`ls -la KDDTrain+.txt`	~18MB
Test file exists	`ls -la KDDTest+.txt`	~3.2MB
Record count (train)	`wc -l KDDTrain+.txt`	125973
Record count (test)	`wc -l KDDTest+.txt`	22544
Column count	`head -1 KDDTrain+.txt | tr ',' '\n' | wc -l`	43 (41 features + label + difficulty)
.gitignore updated	`grep "saved_models" .gitignore`	Dataset patterns present

TASK 5 — Docker Compose Cleanup ⚫

Time Est: 5 min | **Priority:** ⚫ Low

Remove the deprecated `version: "3.8"` key from `docker-compose.yml` per Docker Compose V2 spec.

5.1 Update `docker-compose.yml`

Remove line 2:

```
# BEFORE:
version: "3.8"
```

```
# AFTER:
# (line removed entirely)
```

Verification:

Check	Command	Expected
No version warning	`docker compose config 2>&1`	No deprecation warning
YAML valid	`docker compose config`	No errors
All services present	`docker compose config --services`	postgres, redis, backend, ml-worker, capture

---

TASK 6 — End-to-End Pipeline Verification 🟡

Time Est: 30 min | **Priority:** 🟡 Medium

Verify the complete data pipeline is functioning correctly with the updated feature extractor.

6.1 Pipeline Test on VPS

```
# SSH into VPS
ssh root@187.124.45.161

# 1. Check all services running
docker compose ps
# Expected: postgres, redis, backend, capture — all "Up"
# ml-worker — restarting (expected until Week 3)

# 2. Check capture engine stats
docker compose logs --tail=20 capture
# Expected: Stats lines showing packets, flows, published, pps

# 3. Check Redis pub/sub has messages
docker compose exec redis redis-cli PUBSUB CHANNELS '*'
# Expected: flows:live, system:status

# 4. Check PostgreSQL has flow records
docker compose exec postgres psql -U threatmatrix -d threatmatrix \
  -c "SELECT COUNT(*) FROM network_flows;"
# Expected: > 0 records

# 5. Check feature extraction quality
docker compose exec postgres psql -U threatmatrix -d threatmatrix \
  -c "SELECT features->>'duration', features->>'protocol_type', features-
>>'service', features->>'flag', features->>'src_bytes', features->>'dst_bytes'
FROM network_flows LIMIT 5;"
# Expected: Feature values for each flow
```

```
# 6. Verify new NSL-KDD features present after update
docker compose exec postgres psql -U threatmatrix -d threatmatrix \
  -c "SELECT features->>'land', features->>'urgent', features->>'logged_in',
features->>'count', features->>'dst_host_count' FROM network_flows ORDER BY
created_at DESC LIMIT 5;"
# Expected: Values for new features (may show null for pre-update records)

# 7. Check API returns flow data
curl -s http://localhost:8000/api/v1/flows/stats?interval=1m | python3 -m
json.tool
# Expected: JSON array with flow statistics

# 8. Check capture status
curl -s http://localhost:8000/api/v1/capture/status | python3 -m json.tool
# Expected: {"status": "running", "packets_captured": ..., ...}
```

Verification:

Check	Command	Expected
All services healthy	`docker compose ps`	4 running, 1 restarting
Flows in database	`SELECT COUNT(*)`	> 0 records
Features valid JSONB	`SELECT features`	Valid JSON with 41+ keys
Capture running	`/api/v1/capture/status`	`status: running`
Redis channels active	`PUBSUB CHANNELS`	flows:live present
Stats API returns data	`/api/v1/flows/stats`	Non-empty JSON array
New features present	Check features JSONB	land, count, dst_host_count

---

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   ├── capture/
│   │   ├── engine.py                  🔨 TASK 1 (hardening)
│   │   ├── feature_extractor.py       🔨 TASK 2 (NSL-KDD features +
ConnectionTracker)
│   │   └── publisher.py              🔨 TASK 1 (reconnection logic)
│   ├── ml/                            🆕 TASK 3 (entire directory)
│   │   ├── __init__.py                🆕 TASK 3
│   │   ├── datasets/
│   │   │   ├── __init__.py            🆕 TASK 3
│   │   │   ├── nsl_kdd.py            🆕 TASK 3
│   │   │   └── cicids2017.py         🆕 TASK 3 (stub)
│   │   ├── models/
│   │   │   ├── __init__.py            🆕 TASK 3
│   │   │   ├── isolation_forest.py   🆕 TASK 3 (stub)
```

```
│   │   │   ├── random_forest.py      🆕 TASK 3 (stub)
│   │   │   └── autoencoder.py        🆕 TASK 3 (stub)
│   │   ├── training/
│   │   │   ├── __init__.py            🆕 TASK 3
│   │   │   ├── train_all.py          🆕 TASK 3 (stub)
│   │   │   ├── evaluate.py           🆕 TASK 3 (stub)
│   │   │   └── hyperparams.py        🆕 TASK 3
│   │   ├── inference/
│   │   │   ├── __init__.py            🆕 TASK 3
│   │   │   ├── model_manager.py      🆕 TASK 3 (stub)
│   │   │   ├── ensemble_scorer.py    🆕 TASK 3 (stub)
│   │   │   └── worker.py             🆕 TASK 3 (stub)
│   │   └── saved_models/
│   │       ├── .gitkeep               🆕 TASK 3
│   │       └── datasets/
│   │           ├── KDDTrain+.txt      🆕 TASK 4 (download)
│   │           └── KDDTest+.txt       🆕 TASK 4 (download)
├── docker-compose.yml                 🔨 TASK 5 (remove version)
├── .gitignore                         🔨 TASK 4 (add ML patterns)
└── docs/
    └── worklog/
        └── DAY_08_MAR04.md            🆕 This file
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Command	Expected Result
1	Engine handles malformed packets	Send crafted packet	No crash
2	Multicast filtered	Capture does not include 224.x	Ignored
3	Redis reconnection works	Restart Redis → wait 5s	Auto-reconnects
4	Buffer warning at 80%	Monitor logs under load	Warning message
5	Feature count ≥ 41	`len(features)`	≥ 41
6	NSL-KDD basic features	Check dict keys	9 basic features
7	NSL-KDD content features	Check dict keys	13 content features
8	NSL-KDD time features	ConnectionTracker output	8 time features

#	Verification	Command	Expected Result
9	NSL-KDD host features	ConnectionTracker output	10 host features
10	Land detection	Same src/dst IP+port	`land = True`
11	ML directory complete	`find backend/ml -type f`	18+ files
12	NSL-KDD loader imports	`python -c "from ml.datasets.nsl_kdd import NSLKDDLoader"`	No errors
13	Hyperparams correct	Check values vs PART4	Match exactly
14	NSL-KDD dataset downloaded	`ls backend/ml/saved_models/datasets/`	4 .txt files
15	Train set record count	`wc -l KDDTrain+.txt`	125973
16	Test set record count	`wc -l KDDTest+.txt`	22544
17	Docker version warning gone	`docker compose config 2>&1`	No warning
18	All services running	`docker compose ps`	4 healthy
19	Flows in PostgreSQL	SQL COUNT query	> 0
20	Capture API returns status	`curl /api/v1/capture/status`	Status JSON
21	.gitignore updated	`grep saved_models .gitignore`	Patterns present
22	Python type hints	Code review	All functions typed
23	Async/await used	Code review	All I/O async
24	Error handling	Code review	Try/except blocks

Scope Adherence Verification

Requirement	Source	Verification
41+ features per flow	MASTER_DOC_PART4 §3.1	feature_extractor.py extracts 41+
NSL-KDD feature mapping	MASTER_DOC_PART4 §3.1	Column names match NSL-KDD spec
Content features approximated	MASTER_DOC_PART4 §3.1	Set to 0 (standard practice for network-level capture)

Requirement	Source	Verification
Time-based features (2s window)	MASTER_DOC_PART4 §3.1	ConnectionTracker with 2s window
Host-based features (100 conn)	MASTER_DOC_PART4 §3.1	ConnectionTracker with 100-connection window
ML directory per spec	MASTER_DOC_PART5 §2.1	Matches repo structure exactly
NSL-KDD primary dataset	MASTER_DOC_PART4 §2.1	Downloaded and loader implemented
Hyperparameters from doc	MASTER_DOC_PART4 §4-6	hyperparams.py matches spec
Ensemble weights from doc	MASTER_DOC_PART4 §1.2	0.30 / 0.45 / 0.25
Alert thresholds from doc	MASTER_DOC_PART4 §1.2	0.90 / 0.75 / 0.50 / 0.30
scikit-learn for IF+RF	MASTER_DOC_PART4 §4-5	Import in stubs
TensorFlow for AE	MASTER_DOC_PART4 §6	Import in stubs
Redis reconnection	MASTER_DOC_PART2 §6.1	publisher.py with retry logic

Blockers

Blocker	Severity	Mitigation	Status
No ML models trained	⚪ Medium	Week 3 task — ml-worker will keep restarting	Expected
NSL-KDD content features require DPI	⚪ Medium	Set to 0, standard in modern IDS research	Mitigated
Next.js 16 build error	⚪ Medium	`npm run dev` works for development	Known bug
DEV_MODE enabled on VPS	⚪ Medium	Required for dev, disable before production	Documented

Tomorrow's Preview (Day 9 — Week 2 Day 3)

Per MASTER_DOC_PART5 §3 Week 2 remaining + Week 3 prep:

- Verify NSL-KDD loader with actual dataset (load + preprocess + validate)

- Test ConnectionTracker time/host features against known NSL-KDD records
- Begin ML model wrapper classes (Isolation Forest first)
- Full-Stack Dev: Continue War Room data connection (per FRONTEND_TASKS_DAY8.md)
- Prepare Week 2 end-of-sprint demo (Sunday)

Deferred Tasks (Full-Stack Dev — Separate Document)

The following tasks are tracked in `docs/FRONTEND_TASKS_DAY8.md` and assigned to the Full-Stack Dev:

Task	Priority	Status
DEV_MODE WebSocket connection fallback	⬛	Assigned
Verify API client returns real VPS data	⬛	Assigned
Wire MetricCard to live flow stats	⬛	Assigned
Wire ProtocolChart to live protocol data	⬛	Assigned
Wire TrafficTimeline to live time-series	⬛	Assigned
Wire TopTalkers to live IP rankings	⬜	Assigned
Wire LiveAlertFeed to WebSocket	⬛	Assigned
Wire ThreatMap to WebSocket flow data	⬛	Assigned
Wire ThreatLevel gauge	⬜	Assigned
War Room layout polish per PART3 §2.2	⬜	Assigned

Reference Documents

| Document | Section | Purpose | |----------|---------|---------|| | MASTER_DOC_PART2 | §2.1 | Capture engine architecture | | MASTER_DOC_PART2 | §3.2 | Docker Compose service config | | MASTER_DOC_PART2 | §6.1 | Redis pub/sub architecture | | MASTER_DOC_PART4 | §1.2 | Ensemble scoring strategy + weights | | MASTER_DOC_PART4 | §2.1 | NSL-KDD dataset specification | | MASTER_DOC_PART4 | §2.4 | Dataset preparation pipeline | | MASTER_DOC_PART4 | §3.1 | Feature categories + NSL-KDD mapping | | MASTER_DOC_PART4 | §3.2 | Feature extraction from Scapy | | MASTER_DOC_PART4 | §4.3 | Isolation Forest hyperparameters | | MASTER_DOC_PART4 | §5.2 | Random Forest hyperparameters | | MASTER_DOC_PART4 | §6.3 | Autoencoder hyperparameters | | MASTER_DOC_PART5 | §2.1 | Project structure (ml/ directory) | | MASTER_DOC_PART5 | §3 | Week 2-3 plan | | FRONTEND_TASKS_DAY8.md | Full doc | Deferred frontend tasks |

Task workflow for Day 8 (Week 2 Day 2) — ThreatMatrix AI Sprint 2
Focus: Capture Hardening + NSL-KDD Feature Validation + ML Pipeline Scaffolding
Owner: Lead Architect — Frontend deferred to Full-Stack Dev